

UniApp【萌宠案例】API接口调用与页面渲染实现

本方案基于原有萌宠案例页面布局，移除模拟数据，通过调用真实萌宠API接口实现数据动态渲染。核心内容包括：请求工具类封装（统一处理请求头、异常、拦截器）、首页/分类页/详情页的API调用逻辑、数据渲染与加载状态管理、接口异常处理与用户反馈。全程遵循UniApp跨平台特性，确保在小程序、App、H5端均能稳定运行。

一、前置准备：API接口规范与请求工具类封装

首先明确萌宠API的接口规范，再封装通用请求工具类，简化接口调用流程，统一处理重复逻辑（如token携带、超时设置、错误提示）。

1. 萌宠API接口规范（示例）

假设后端提供的萌宠API遵循RESTful规范，基础信息如下：

接口名称	请求地址	请求方法	请求参数
首页数据（轮播+推荐）	https://api.pet.com/home	GET	无
萌宠分类列表	https://api.pet.com/category	GET	categoryId（分类选）、page（页码（每页条数）
萌宠详情	https://api.pet.com/pet/:id	GET	id（萌宠ID，路径
点赞/取消点赞	https://api.pet.com/pet/:id/like	POST	id（萌宠ID，路径
收藏/取消收藏	https://api.pet.com/pet/:id/collect	POST	id（萌宠ID，路径



实际开发中需替换为真实接口地址与参数，确保后端接口已配置CORS跨域（H5端）、请求白名单（小程序端）。

2. 通用请求工具类封装 (utils/request.js)

封装uni.request，实现请求拦截（添加token、统一请求头）、响应拦截（解析数据、统一错误处理），减少页面重复代码。

Code block

```
1  /**
2   * 通用请求工具类
3   * @param {Object} options - 请求配置
4   * @returns {Promise} - 响应结果
5   */
6  const request = (options) => {
7    // 1. 基础配置
8    const baseUrl = 'https://api.pet.com' // 基础API地址
9    const token = uni.getStorageSync('token') // 从缓存获取token
10
11    // 2. 合并请求配置
12    const config = {
13      url: baseUrl + options.url,
14      method: options.method || 'GET',
15      header: {
16        'Content-Type': 'application/json',
17        'Authorization': token ? `Bearer ${token}` : '', // 携带身份令牌
18        ...options.header // 允许覆盖默认请求头
19      },
20      data: options.data || {},
21      timeout: options.timeout || 10000, // 超时时间10秒
22      sslVerify: options.sslVerify !== false, // 生产环境默认验证SSL证书
23      ...options
24    }
25
26    // 3. 返回Promise, 处理请求结果
27    return new Promise((resolve, reject) => {
28      // 4. 发起请求
29      uni.request({
30        ...config,
31        // 成功回调
32        success: (res) => {
33          const { statusCode, data } = res
34
35          // 5. 处理HTTP状态码
36          if (statusCode >= 200 && statusCode < 300) {
37            // 6. 处理业务状态码
38            if (data.code === 0) {
```

```

39         // 业务成功: 返回响应数据
40         resolve(data.data)
41     } else {
42         // 业务错误: 提示错误信息
43         uni.showToast({ title: data.msg || '业务请求失败', icon: 'error' })
44         reject(new Error(data.msg || '业务请求失败'))
45     }
46 } else if (statusCode === 401) {
47     // 401: 未登录/ token过期, 跳转登录页
48     uni.showToast({ title: '请先登录', icon: 'none' })
49     uni.navigateTo({ url: '/pages/login/login' })
50     reject(new Error('未授权访问'))
51 } else if (statusCode === 404) {
52     uni.showToast({ title: '接口不存在', icon: 'error' })
53     reject(new Error('接口不存在'))
54 } else if (statusCode === 500) {
55     uni.showToast({ title: '服务器异常', icon: 'error' })
56     reject(new Error('服务器异常'))
57 } else {
58     uni.showToast({ title: `请求错误: ${statusCode}`, icon: 'error' })
59     reject(new Error(`请求错误: ${statusCode}`))
60 }
61 },
62 // 失败回调 (网络错误、超时等)
63 fail: (err) => {
64     let errMsg = '网络请求失败, 请重试'
65     if (err.errCode === 1001) errMsg = '无网络, 请检查网络设置'
66     if (err.errCode === 1004) errMsg = '请求超时, 请稍后再试'
67
68     uni.showToast({ title: errMsg, icon: 'error' })
69     reject(new Error(errMsg))
70 },
71 // 完成回调: 关闭加载动画 (若页面开启)
72 complete: () => {
73     if (options.showLoading !== false) {
74         uni.hideLoading()
75     }
76 }
77 })
78 })
79 }
80
81 // 7. 简化GET/POST请求调用
82 request.get = (url, data = {}, options = {}) => {
83     return request({ url, method: 'GET', data, ...options })
84 }
85

```

```
86 request.post = (url, data = {}, options = {}) => {
87   return request({ url, method: 'POST', data, ...options })
88 }
--
```

二、首页：调用首页API渲染轮播与推荐数据

首页通过调用/home接口获取轮播图、分类快捷入口、萌宠推荐等数据，结合加载状态管理，实现数据加载→渲染→异常处理的完整流程。

1. 页面结构（pages/index/index.vue）

保留原有布局结构，新增加载状态提示（如“加载中...”），优化数据为空时的展示。

Code block

```
1  <template>
2    <view class="index-page">
3
4      <view class="loading-overlay" v-if="isLoading">
5        <uni-loading-icon type="spin" size="40" color="#FF9A6C"></uni-loading-
6        icon>
7        <text class="loading-text">加载中...</text>
8      </view>
9
10     <view class="swiper-container" v-if="swiperList.length">
11       <swiper indicator-dots autoplay circular interval="3000" duration="500">
12         <swiper-item v-for="(item, index) in swiperList" :key="item.id">
13           <image :src="item.imgUrl" mode="widthFix" class="swiper-img"></image>
14         </swiper-item>
15       </swiper>
16     </view>
17
18
19     <view class="category-shortcut pet-card" v-if="categoryList.length">
20       <view class="shortcut-item" v-for="item in categoryList" :key="item.id"
21       @click="toCategory(item.id)">
22         <view class="item-icon" :style="{backgroundColor: item.color}">
23           <image :src="item.icon" mode="widthFix" class="icon-img"></image>
24         </view>
25         <view class="item-text">{{ item.name }}</view>
26       </view>
27
28
29     <view class="pet-recommend" v-if="recommendList.length">
30       <view class="section-header">
```

```

31         <text class="header-title">萌宠推荐</text>
32         <navigator url="/pages/category/category" class="more-link">查看更多
</navigator>
33     </view>
34     <view class="recommend-list">
35         <navigator
36             class="pet-item pet-card"
37             v-for="pet in recommendList"
38             :key="pet.id"
39             :url="'/pages/detail/detail?id='+pet.id"
40         >
41             <image :src="pet.coverImg" mode="widthFix" class="pet-img"></image>
42             <view class="pet-info">
43                 <view class="pet-name">{{ pet.name }}</view>
44                 <view class="pet-breed">品种: {{ pet.breed }}</view>
45                 <view class="pet-meta">
46                     <view class="like-count"><image src="/static/icon/like.png"
mode="widthFix" class="like-icon"></image> {{ pet.likeCount }}</view>
47                     <view class="view-count">浏览 {{ pet.viewCount }}</view>
48                 </view>
49             </navigator>
50         </view>
51     </view>
52 </view>
53
54
55     <view class="empty-tip" v-if="!isLoading && (!swiperList.length &&
!recommendList.length)">
56         <image src="/static/icon/empty.png" mode="widthFix" class="empty-img">
</image>
57         <text class="empty-text">暂无萌宠数据~</text>
58     </view>
59 </view>
60 </template>

```

2. API调用与数据渲染逻辑

Code block

```

1  <script setup>
2  import { ref, onLoad } from 'vue'
3  import { useRouter } from 'vue-router'
4  import request from '../utils/request' // 导入请求工具类
5
6  const router = useRouter()
7

```

```

8 // 页面状态与数据
9 const isLoading = ref(true) // 全局加载状态
10 const swiperList = ref([]) // 轮播图数据
11 const categoryList = ref([]) // 分类快捷入口数据
12 const recommendList = ref([]) // 萌宠推荐数据
13
14 // 页面加载时调用API获取数据
15 onLoad(async () => {
16   try {
17     // 显示加载动画（请求工具类会自动关闭）
18     uni.showLoading({ title: '加载萌宠数据...', mask: true })
19
20     // 调用首页API
21     const homeData = await request.get('/home', {}, { showLoading: false })
22     // 解构API返回数据（与后端响应格式对应）
23     const { swiper, category, recommend } = homeData
24
25     // 赋值数据，触发页面渲染
26     swiperList.value = swiper || []
27     categoryList.value = category || []
28     recommendList.value = recommend || []
29   } catch (err) {
30     console.error('首页数据加载失败: ', err)
31   } finally {
32     // 关闭加载状态
33     isLoading.value = false
34   }
35 })
36
37 // 跳转到分类页（携带分类ID）
38 const toCategory = (id) => {
39   router.push({ path: '/pages/category/category', query: { categoryId: id } })
40 }
41 </script>

```

3. 样式补充（scoped）

Code block

```

1 /* 加载中覆盖层 */
2 .loading-overlay {
3   position: fixed;
4   top: 0;
5   left: 0;
6   width: 100%;
7   height: 100%;

```

```
8     background-color: rgba(255, 255, 255, 0.8);
9     display: flex;
10    flex-direction: column;
11    align-items: center;
12    justify-content: center;
13    z-index: 999;
14  }
15  .loading-text {
16    font-size: 32rpx;
17    color: var(--primary-color);
18    margin-top: 20rpx;
19  }
20
21  /* 数据为空提示 */
22  .empty-tip {
23    width: 100%;
24    padding: 100rpx 0;
25    display: flex;
26    flex-direction: column;
27    align-items: center;
28    justify-content: center;
29  }
30  .empty-img {
31    width: 200rpx;
32    height: 200rpx;
33    margin-bottom: 30rpx;
34  }
35  .empty-text {
36    font-size: 32rpx;
37    color: var(--light-text);
38  }
39
40  /* 原有样式保持不变，此处省略... */
```

三、分类页：调用分类API实现分页渲染

分类页通过调用/category接口，结合分页参数（page、size）实现下拉刷新、上拉加载更多功能，支持按分类ID筛选数据。

1. 页面结构与逻辑（pages/category/category.vue）

Code block

```
1  <template>
2    <view class="category-page">
```

```

3
4     <view class="category-tabs">
5         <scroll-view scroll-x class="tabs-scroll">
6             <view
7                 class="tab-item"
8                 v-for="tab in categoryTabs"
9                 :key="tab.id"
10                :class="{ active: activeTab === tab.id }"
11                @click="switchTab(tab.id)"
12            >
13                {{ tab.name }}
14            </view>
15        </scroll-view>
16    </view>
17
18
19    <scroll-view
20        class="category-list"
21        scroll-y
22        refresher-enabled
23        refresher-threshold="40"
24        @refresherrefresh="onRefresh"
25        @scrolltolower="onLoadMore"
26    >
27
28        <view class="list-loading" v-if="isRefreshing">
29            <uni-loading-icon type="spin" size="30" color="#FF9A6C"></uni-loading-
30            icon>
31        </view>
32
33        <view class="pet-item pet-card" v-for="pet in petList" :key="pet.id">
34            <navigator :url="'/pages/detail/detail?id='+pet.id" class="pet-link">
35                <image :src="pet.coverImg" mode="widthFix" class="pet-img"></image>
36                <view class="pet-info">
37                    <view class="pet-name">{{ pet.name }}</view>
38                    <view class="pet-desc">{{ pet.desc }}</view>
39                    <view class="pet-like">
40                        <image src="/static/icon/like.png" mode="widthFix" class="like-
41                        icon"></image>
42                        <text class="like-count">{{ pet.likeCount }}</text>
43                    </view>
44                </view>
45            </navigator>
46        </view>
47

```



```

48     <view class="empty-tip" v-if="!isLoading && !isRefreshing &&
petList.length === 0">
49         <image src="/static/icon/empty.png" mode="widthFix" class="empty-img">
</image>
50         <text class="empty-text">该分类暂无萌宠~</text>
51     </view>
52
53
54     <view class="load-tip" v-if="petList.length > 0">
55         <uni-loading-icon type="spin" size="24" color="#FF9A6C" v-
if="isLoadingMore"></uni-loading-icon>
56         <text v-else>已加载全部</text>
57     </view>
58 </scroll-view>
59 </view>
60 </template>

```

2. API调用与分页逻辑

Code block

```

1  <script setup>
2  import { ref, onLoad, onReady } from 'vue'
3  import request from '../utils/request'
4
5  // 页面状态与分页参数
6  const activeTab = ref(0) // 当前选中分类ID (0为全部)
7  const categoryTabs = ref([]) // 分类标签数据 (从API获取)
8  const petList = ref([]) // 萌宠列表数据
9  const page = ref(1) // 当前页码
10 const pageSize = ref(10) // 每页条数
11 const total = ref(0) // 总数据量
12 const isRefreshing = ref(false) // 下拉刷新状态
13 const isLoadingMore = ref(false) // 上拉加载状态
14 const isNoMore = ref(false) // 是否无更多数据
15
16 // 页面加载：获取分类标签与初始数据
17 onLoad(async (options) => {
18     try {
19         // 1. 先获取分类标签数据
20         await getCategoryTabs()
21
22         // 2. 若从首页跳转，选中对应分类
23         if (options.categoryId) {
24             activeTab.value = Number(options.categoryId)
25         }

```

```
26
27     // 3. 加载初始数据
28     await getPetList()
29   } catch (err) {
30     console.error('分类页初始化失败: ', err)
31   }
32 })
33
34 // 获取分类标签数据
35 const getCategoryTabs = async () => {
36   try {
37     const data = await request.get('/category/tabs')
38     // 拼接“全部”分类在最前面
39     categoryTabs.value = [{ id: 0, name: '全部' }, ...(data || [])]
40   } catch (err) {
41     console.error('分类标签加载失败: ', err)
42     // 异常时使用默认分类
43     categoryTabs.value = [{ id: 0, name: '全部' }, { id: 1, name: '猫咪' }, {
44       id: 2, name: '狗狗' }]
45   }
46
47   // 调用API获取萌宠列表（支持分页与分类筛选）
48   const getPetList = async () => {
49     // 若正在加载或无更多数据，阻止重复请求
50     if ((isRefreshing.value || isLoadingMore.value) || isNoMore.value) return
51
52     // 设置加载状态
53     if (isRefreshing.value) {
54       page.value = 1 // 下拉刷新重置页码
55     } else {
56       isLoadingMore.value = true
57     }
58
59     try {
60       // 构造请求参数
61       const params = {
62         page: page.value,
63         size: pageSize.value
64       }
65       // 若不是“全部”分类，添加分类ID参数
66       if (activeTab.value !== 0) {
67         params.categoryId = activeTab.value
68       }
69
70       // 调用分类列表API
71       const data = await request.get('/category', params)
```

```
72     const { list, total: totalCount } = data
73     total.value = totalCount
74
75     // 处理数据：下拉刷新覆盖数据，上拉加载追加数据
76     if (isRefreshing.value) {
77       petList.value = list || []
78     } else {
79       petList.value = [...petList.value, ...(list || [])]
80     }
81
82     // 判断是否无更多数据
83     isNoMore.value = petList.value.length >= total.value
84   } catch (err) {
85     console.error('萌宠列表加载失败: ', err)
86   } finally {
87     // 关闭加载状态
```

四、详情页：调用详情API与交互接口

详情页通过萌宠ID调用/pet/:id接口获取详情数据，同时实现点赞、收藏等交互功能（调用对应API），实时更新页面状态。

1. 页面结构与逻辑（pages/detail/detail.vue）

Code block

```
1  <template>
2    <view class="detail-page">
3
4      <view class="loading-overlay" v-if="isLoading">
5        <uni-loading-icon type="spin" size="40" color="#FF9A6C"></uni-loading-
6        icon>
7        <text class="loading-text">加载中...</text>
8      </view>
9
10     <view class="pet-banner" v-if="petInfo.id">
11       <swiper indicator-dots circular autoplay interval="5000">
12         <swiper-item v-for="(img, index) in petImgList" :key="index">
13           <image :src="img" mode="widthFix" class="banner-img"></image>
14         </swiper-item>
15       </swiper>
16     </view>
17
18
19     <view class="pet-base-info pet-card" v-if="petInfo.id">
20       <view class="pet-name-bar">
```

```
21     <view class="pet-name">{{ petInfo.name }}</view>
22     <view class="pet-actions">
23         <view class="action-btn" @click="handleLike" :class="{ liked:
petInfo.isLiked }">
24             <image src="/static/icon/like.png" mode="widthFix" class="action-
icon"></image>
25             <text class="action-text">{{ petInfo.isLiked ? '已喜欢' : '喜欢' }}
</text>
26             <text class="like-count">{{ petInfo.likeCount }}</text>
27         </view>
28         <view class="action-btn" @click="handleCollect" :class="{ collected:
petInfo.isCollected }">
29             <image src="/static/icon/collect.png" mode="widthFix"
class="action-icon"></image>
30             <text class="action-text">{{ petInfo.isCollected ? '已收藏' : '收
藏' }}</text>
31         </view>
32     </view>
33 </view>
34 <view class="pet-meta">
35     <view class="meta-item">品种: {{ petInfo.breed }}</view>
36     <view class="meta-item">年龄: {{ petInfo.age }}</view>
37     <view class="meta-item">性格: {{ petInfo.character }}</view>
38 </view>
39 <view class="pet-desc">{{ petInfo.desc }}</view>
40 </view>
41
42
43 <view class="feeding-tips pet-card" v-if="feedingTips.length">
44     <view class="section-title">饲养要点</view>
45     <view class="tips-list">
46         <view class="tips-item" v-for="(tip, index) in feedingTips"
:key="index">
47             <view class="tip-icon">{{ index + 1 }}</view>
48             <view class="tip-content">{{ tip }}</view>
49         </view>
50     </view>
51 </view>
52
53
54 <view class="related-recommend" v-if="relatedList.length">
55     <view class="section-title">相关推荐</view>
56     <view class="related-list">
57         <navigator
58             class="related-item pet-card"
59             v-for="pet in relatedList"
60             :key="pet.id">
```

```

61         :url="'/pages/detail/detail?id='+pet.id"
62     >
63     <image :src="pet.coverImg" mode="widthFix" class="related-img">
</image>
64     <view class="related-name">{{ pet.name }}</view>
65     <view class="related-breed">{{ pet.breed }}</view>
66     </navigator>
67 </view>
68 </view>
69
70
71 <view class="error-tip" v-if="!isLoading && !petInfo.id">
72     <image src="/static/icon/error.png" mode="widthFix" class="error-img">
</image>
73     <text class="error-text">萌宠详情加载失败</text>
74     <button class="retry-btn" @click="reloadDetail">重试</button>
75 </view>
76
77
78 <view class="bottom-bar" v-if="petInfo.id">

```

2. API调用与交互逻辑

Code block

```

1  <script setup>
2  import { ref, onLoad } from 'vue'
3  import request from '../utils/request'
4
5  // 页面状态与数据
6  const petId = ref('') // 萌宠ID (从路由获取)
7  const isLoading = ref(true) // 加载状态
8  const petInfo = ref({}) // 萌宠基本信息
9  const petImgList = ref([]) // 萌宠图片列表
10 const feedingTips = ref([]) // 饲养要点
11 const relatedList = ref([]) // 相关推荐
12
13 // 页面加载: 获取萌宠ID并加载详情
14 onLoad(async (options) => {
15     if (!options.id) {
16         uni.showToast({ title: '萌宠ID不存在', icon: 'error' })
17         uni.navigateBack()
18         return
19     }
20     petId.value = options.id
21     await reloadDetail()
22 })

```

```
23
24 // 重新加载萌宠详情
25 const reloadDetail = async () => {
26   isLoading.value = true
27   try {
28     // 调用萌宠详情API (路径参数: petId)
29     const data = await request.get(`/pet/${petId.value}`)
30     // 解构数据并赋值
31     petInfo.value = data.petInfo || {}
32     petImgList.value = data.petImgList || []
33     feedingTips.value = data.feedingTips || []
34     relatedList.value = data.relatedList || []
35   } catch (err) {
36     console.error('萌宠详情加载失败: ', err)
37   } finally {
38     isLoading.value = false
39   }
40 }
41
42 // 处理点赞/取消点赞
43 const handleLike = async () => {
44   try {
45     // 调用点赞API
46     const data = await request.post(`/pet/${petId.value}/like`)
47     // 实时更新页面状态 (无需重新请求详情)
48     petInfo.value.isLiked = data.isLiked
49     petInfo.value.likeCount = data.likeCount
50     uni.showToast({ title: data.isLiked ? '点赞成功' : '取消点赞' })
51   } catch (err) {
52     console.error('点赞操作失败: ', err)
53   }
54 }
55
56 // 处理收藏/取消收藏
57 const handleCollect = async () => {
58   try {
59     // 调用收藏API
60     const data = await request.post(`/pet/${petId.value}/collect`)
61     // 实时更新页面状态
62     petInfo.value.isCollected = data.isCollected
63     uni.showToast({ title: data.isCollected ? '收藏成功' : '取消收藏' })
64   } catch (err) {
65     console.error('收藏操作失败: ', err)
66   }
67 }
68
69 // 分享萌宠
```

```
70 const sharePet = () => {
71   if (!petInfo.value.id) return
72   uni.share({
73     title: `超可爱的${petInfo.value.breed}: ${petInfo.value.name}`,
74     imageUrl: petImgList.value[0] || '/static/default-pet.png',
75     path: `/pages/detail/detail?id=${petId.value}`,
76     success: () => {
77       uni.showToast({ title: '分享成功' })
78     }
79   })
80 }
81 </script>
```

五、API调用核心注意事项

1. 数据状态管理

- **加载状态全覆盖**：每个API请求都需配合加载状态（如全屏加载、列表加载），避免用户重复操作，提升体验。
- **数据为空处理**：所有列表页面都需添加数据为空提示（如“暂无萌宠数据”），避免页面空白，引导用户理解当前状态。
- **状态实时更新**：交互类API（点赞、收藏）调用成功后，直接通过响应数据更新页面状态，无需重新请求详情，提升响应速度。

2. 异常处理与容错

- **网络异常兜底**：请求工具类已统一处理网络错误（如无网络、超时），页面无需重复处理，只需关注业务异常。
- **接口异常降级**：分类标签等非核心数据加载失败时，提供默认数据（如固定分类列表），保证页面基本功能可用。
- **重试机制**：详情页等核心页面提供“重试”按钮，允许用户在加载失败后重新发起请求。

3. 跨平台兼容性

- **小程序域名白名单**：微信小程序需在“开发设置”中添加API域名到request合法域名列表，否则请求会被拦截。
- **H5端跨域处理**：开发环境通过manifest.json的h5.devServer配置代理，生产环境需后端配置CORS响应头（Access-Control-Allow-Origin）。
- **App端SSL验证**：生产环境需确保API使用HTTPS，App端默认验证SSL证书，自签名证书需在manifest.json中配置sslVerify为false（仅开发环境使用）。

4. 性能优化

- **避免重复请求：**分页加载时添加状态锁（isLoadingMore），防止用户快速滑动触发多次请求。
- **数据缓存策略：**首页轮播图、分类标签等高频访问且变化不频繁的数据，可添加本地缓存（uni.setStorageSync），减少重复请求。
- **图片懒加载：**列表图片添加lazy-load属性（如<image lazy-load :src="pet.coverImg">），提升页面加载速度，减少流量消耗。

六、总结

本方案通过封装通用请求工具类，实现了萌宠案例中所有页面的API调用与数据渲染，核心优势在于：

1. **代码复用性高：**请求工具类统一处理请求头、异常、加载状态，页面只需关注业务逻辑，减少重复代码。
2. **用户体验流畅：**完善的加载状态、空数据提示、异常重试机制，让用户清晰感知操作结果。
3. **跨平台兼容性好：**针对小程序、App、H5端的API调用差异做了适配，确保全平台稳定运行。
4. **可扩展性强：**新增接口（如评论、视频）时，可直接复用现有请求逻辑，快速集成到页面中。

实际开发中，需根据后端真实API的响应格式调整数据解构逻辑，同时结合项目需求扩展功能（如添加评论区、萌宠视频播放等），保持现有代码框架即可实现快速迭代。

（注：文档部分内容可能由 AI 生成）